MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Database Systems Lab
Mini project:
College Quiz Management System

Final Project Report

A desktop app to manage online quizzes in colleges

Submitted By:

Name	*Roll No*	*Reg No*	*Class*
Shreeniketh E R	*10*	*240953148*	*CCE-B*
Arnav Varshney	*31*	*240953382*	*CCE-B*
Ankitha M	*16*	*240953382*	*CCE-B*

School of Computer Engineering

Manipal Institute of Technology, Manipal.

1st April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

School of Computer Engineering

CERTIFICATE

This is to certify that the project titled College Quiz Management System is a record of the bonafide work done by Shreeniketh E R(240953148), Ankitha M (240953194) and Arnav Varshney(240953382) submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in COMPUTER AND COMMUNICATION ENGINEERING of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

1. **Dr. Chitra K, Assistant Professor, SCE.**

2. **Dr. Anita S Kini, Assistant Professor, SC**E.

Index

This index lists the main report sections. Word can update the table of contents field if you want live page numbers after opening the document.

Index 1

Abstract 4

Chapter 1: Introduction 4

- 1.1 Background 4
- 1.2 Purpose 4
- 1.3 Scope 5

Chapter 2: Problem Statement 5

- 2.1 Context and Motivation 5
- 2.2 Data Requirements 5
- 2.3 Functional Requirements 5

Chapter 3: Methodology 5

- 3.1 Requirement Analysis 5
- 3.2 System Design 6
- 3.3 Database Implementation 6
- 3.4 Development 6
- 3.5 Testing and Deployment 6

Chapter 4: ER Diagram and Relational Tables 6

- 4.1 Entity Summary 6
- 4.2 Relationships 6
- 4.3 ER Diagram 7
- 4.4 Sample Data 8

Chapter 5: DDL Commands 8

Chapter 6: SQL Queries 9

- 6.1 Data Retrieval 9
- 6.2 Aggregate and Analytical Queries 9
- 6.3 Admin Operational Query 9
- 6.4 DML Statements 9

Chapter 7: PL/SQL Procedures, Functions, and Triggers 9

Chapter 8: Java Backend Layer 10

Chapter 9: JavaFX Frontend Layer 11

Chapter 10: Screens, Workflows, and Code Snippets 11

Chapter 11: Validation, Security, and Data Integrity 13

Chapter 12: Testing and Deployment 13

Chapter 13: Conclusion 14

Sources ..... **Error! Bookmark not defined.**

Abstract

The College Quiz Management System is a desktop application that supports the full life cycle of a college quiz. It provides role-based access for admins, teachers, and students, and it uses Oracle Database, Java, and JavaFX to keep data storage, business logic, and presentation clearly separated.

The system is designed around a practical academic workflow. Teachers create subjects, quizzes, and questions; students take timed quizzes with autosave; and teachers review and publish results manually. This makes the project suitable for demonstrating database design, PL/SQL logic, JDBC transaction handling, and JavaFX screen navigation in one integrated application.

Field	Value
Project	College Quiz Management System
Architecture	Oracle PL/SQL + Java backend + JavaFX frontend
Users	Admin, teacher, student
Database	Oracle SQL*Plus / PL/SQL
Backend	Java 17, JDBC, DAO, Service
Frontend	JavaFX 21, FXML, CSS
Build Tool	Maven

Chapter 1: Introduction

1.1 Background

Quiz systems are widely used in academic environments because they can standardize assessment, reduce manual marking effort, and support immediate data collection. This project focuses on a controlled desktop implementation rather than a web-based or mobile platform so that the relationship between the database, the backend, and the UI remains easy to study.

The application uses a layered approach because each layer is responsible for a different concern. The Oracle database stores the authoritative data and enforces the rules. The Java backend wraps those rules in transactions and exposes them through service methods. The JavaFX frontend gives the user a straightforward interface that hides SQL and procedural details.

1.2 Purpose

- Support quiz administration for college users.
- Keep user roles separated and easy to navigate.
- Store attempts and results in a reliable database structure.
- Demonstrate a complete Java and Oracle integration pattern.

1.3 Scope

The scope includes all core quiz management functions: login, department and user management, subject and quiz creation, question entry, timed attempts, autosave, scoring, publishing, and result viewing. It does not attempt to add every advanced learning-management feature, because the goal is a clear and dependable project rather than an overgrown one.

Chapter 2: Problem Statement

2.1 Context and Motivation

In a college setting, quiz creation and result handling can become repetitive and error-prone if they are managed manually. A centralized application reduces duplicate effort and makes it easier to enforce rules such as unique attempts, correct option counts, and controlled result publication.

2.2 Data Requirements

The project requires structured storage for departments, teachers, students, subjects, quizzes, questions, question options, attempts, answers, and publication logs. Each entity has a specific role in the quiz workflow, and the relationships between them must be stable to keep the application trustworthy.

Entity	Role in the System	Typical Data
Department	Groups teachers, students, and subjects	Code, name
Teacher	Authors quizzes and publishes results	Teacher code, password, department
Student	Takes quizzes and views published results	Registration number, password, department
Quiz	Defines a timed assessment	Title, duration, date, total marks
Question	Represents one test item	Question text, marks, display order
Question option	Stores the answer choices	Option text, correctness flag

2.3 Functional Requirements

- Allow role-based login for admin, teacher, and student.
- Allow teachers to create subjects, quizzes, questions, and options.
- Allow students to start timed attempts and autosave answers.
- Allow teachers to review and publish results manually.
- Allow students to view results only after publication.

Chapter 3: Methodology

3.1 Requirement Analysis

The requirement analysis phase identified three distinct user groups and a shared set of data rules. The main challenge was making the application usable for each role without exposing irrelevant functions. That is why the screen flow is role-specific and the database enforces the quiz rules instead of leaving them to the UI.

3.2 System Design

The system was designed as a three-tier desktop application. The design goal was to separate data storage, business rules, and presentation as much as possible. The DAO and service layers keep JDBC work out of the controllers, and the controller layer focuses on interaction instead of SQL.

3.3 Database Implementation

The database implementation uses normalized tables, constraints, triggers, views, and PL/SQL packages. This means the schema is not just a place to store rows. It actively participates in the behavior of the application by enforcing the publish rules, autosave flow, and single-choice quiz model.

```
public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(DbConfig.url(), DbConfig.user(),
DbConfig.password());
}
```

3.4 Development

Development followed the natural build order of the project. The Oracle schema was created first, then the Java backend was added, and finally the JavaFX screens were wired on top. This sequence made it easier to keep each layer consistent with the one below it.

3.5 Testing and Deployment

Testing focuses on whether the app launches, logs in correctly, creates valid data, handles attempts safely, and displays published results only when expected. Deployment is straightforward because the project is Maven-based and the database setup script can be rerun when needed.

Chapter 4: ER Diagram and Relational Tables

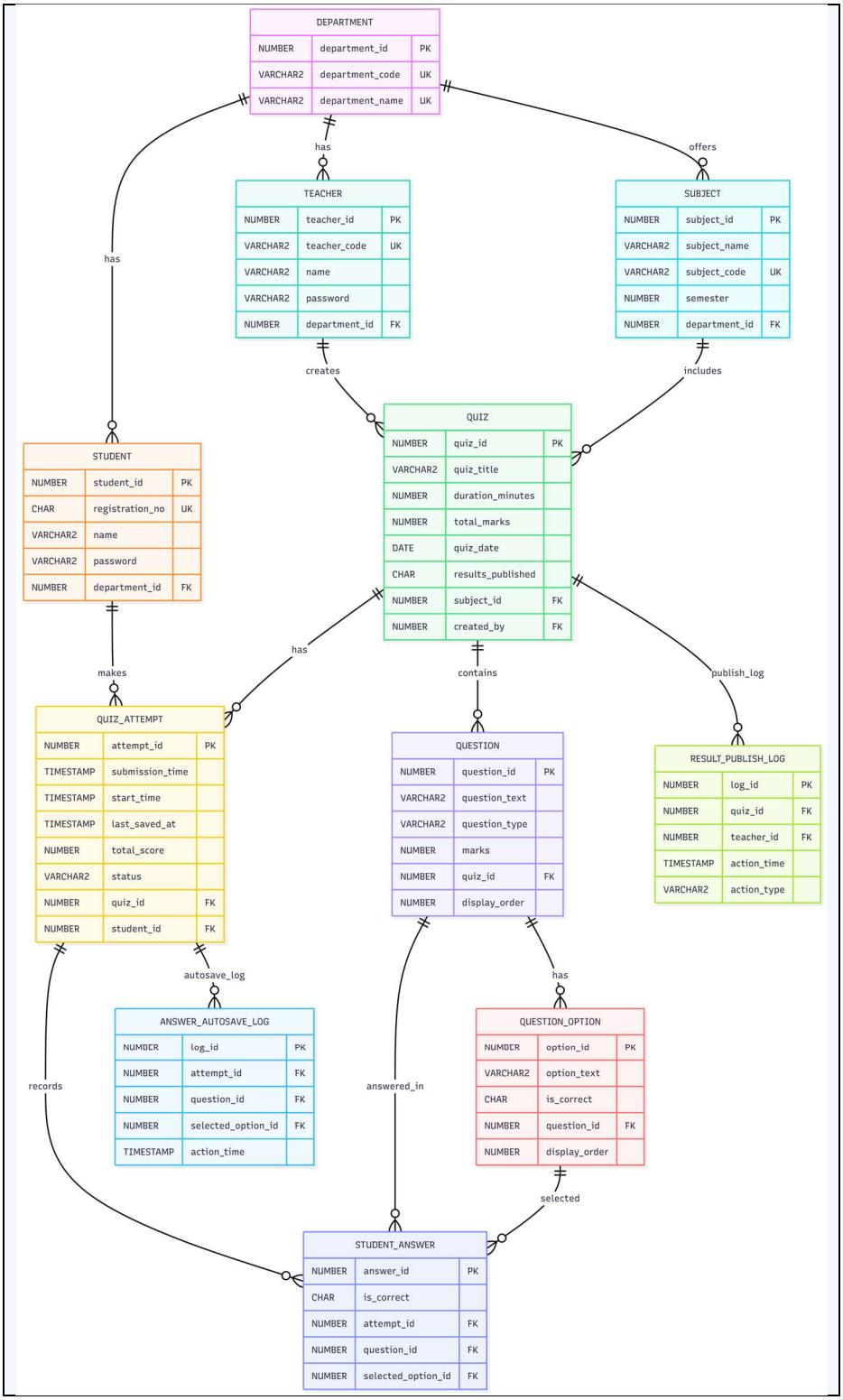
4.1 Entity Summary

The relational model centers on a handful of clear entities. Department organizes people and subjects. Teacher and student provide role identities. Subject links academic context to quiz content. Quiz, question, and question option model the actual assessment. Attempt and answer track student participation.

4.2 Relationships

The relationships are straightforward and follow the quiz workflow. A department has many teachers and students. A subject belongs to one department. A quiz belongs to one subject. A quiz has many questions. A question has many options. A student has many attempts over time, but only one attempt per quiz is allowed by the business rules.

4.3 ER Diagram



4.4 Sample Data

The seed data creates a small but complete testing environment. It includes departments, teachers, students, subjects, quizzes, and questions so that dashboards and quiz workflows can be demonstrated without manual data entry.

Table	Example Contents	Why It Matters
Department	3 departments	Provides the academic structure
Teacher	3 teachers	Lets the teacher flow be tested
Student	3 students	Supports login and quiz attempts
Subject	4 subjects	Gives quizzes a category
Quiz	4 quizzes	Provides sample assessments
Question	20 questions	Makes the quiz screens usable
Screenshot Placeholder: Seeded data view **Insert a screenshot showing populated tables or the SQL*Plus output after running the seed script.**		

Chapter 5: DDL Commands

The DDL portion of the project creates the Oracle tables, sequences, constraints, triggers, packages, and views. These commands define the structure and the behavior of the system. The tables store the data, while triggers and packages enforce the runtime rules.

A good example of the application design is the teacher dashboard refresh logic. The UI asks the service layer for dashboard rows, and the DAO returns projection data that the controller can place directly into the table view.

```
var rows = service.getTeacherDashboard(AppSession.getLoggedInTeacherId());
dashboardTable.setItems(FXCollections.observableArrayList(rows));
```

This is a useful design pattern because it keeps the controller readable. The controller does not know how the rows were built. It only knows how to display them and respond to user actions.

DDL Category	Used For	Effect
Tables	Core persistent entities	Store quiz and account data
Sequences	Primary-key generation	Produce stable identifiers
Triggers	Validation and derived values	Keep the schema consistent
Packages	Quiz workflows	Encapsulate complex rules
Views	Dashboards and results	Simplify reporting queries

```
CREATE TABLE DEPARTMENT (
   DEPARTMENT_ID    NUMBER(10)     NOT NULL,
   DEPARTMENT_CODE  VARCHAR2(10)   NOT NULL,
   DEPARTMENT_NAME  VARCHAR2(100)  NOT NULL,
   CONSTRAINT PK_DEPARTMENT PRIMARY KEY (DEPARTMENT_ID),
   CONSTRAINT UQ_DEPARTMENT_CODE UNIQUE (DEPARTMENT_CODE),
   CONSTRAINT UQ_DEPARTMENT_NAME UNIQUE (DEPARTMENT_NAME),
   CONSTRAINT CHK_DEPARTMENT_CODE_NN CHECK (TRIM(DEPARTMENT_CODE) IS NOT NULL),
   CONSTRAINT CHK_DEPARTMENT_NAME_NN CHECK (TRIM(DEPARTMENT_NAME) IS NOT NULL)
);
```

Chapter 6: SQL Queries

6.1 Data Retrieval

The DAO interface exposes methods for fetching departments, teachers, students, subjects, quizzes, questions, and options. These retrieval methods are the foundation of the UI because every screen depends on loading the current data from Oracle before it can display anything meaningful.

6.2 Aggregate and Analytical Queries

Teacher and student dashboards rely on summary-style data rather than raw row-by-row content. That is why the project uses projection models such as TeacherDashboardRow and StudentResultRow. They keep the UI simple and let the database do the heavy lifting for summary views.

6.3 Admin Operational Query

Administrative screens need clear and reliable actions such as deleting records or clearing operational data. The DAO includes methods for cleanup and removal tasks, and the UI uses confirmation dialogs so destructive actions are intentional rather than accidental.

6.4 DML Statements

The main insert and update operations include subject creation, quiz creation, question insertion, option insertion, autosaving answers, clearing answers, publishing results, and unpublishing results. These are the core business actions of the application because they change the state of the quiz system.

```
public Integer createDepartment(Connection connection, String departmentCode, String departmentName)
        throws SQLException {
    int departmentId = nextSequenceValue(connection, "SEQ_DEPARTMENT");
    String sql = """
        INSERT INTO DEPARTMENT (DEPARTMENT_ID, DEPARTMENT_CODE, DEPARTMENT_NAME)
        VALUES (?, UPPER(TRIM(?)), ?)
        """;
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, departmentId);
        ps.setString(2, departmentCode);
        ps.setString(3, departmentName);
        ps.executeUpdate();
    }
    return departmentId;
  }
```

Chapter 7: PL/SQL Procedures, Functions, and Triggers

The PL/SQL layer is the heart of the project's business logic. It handles attempt creation, autosave, scoring, publication checks, and the validation of quiz completeness. This keeps the application reliable even if the Java layer changes later.

```
public static void showTeacherReview() {
    loadAndShow("/com/collegequiz/view/teacher-review.fxml", "Teacher Review", 1240,
780);
}
```

The review screen is useful because it combines dashboard information with publishing controls. In the controller, the publish button is disabled until a valid selection exists and the quiz has at least one attempt. That kind of logic is important because it prevents invalid actions before they reach the database.

```
if (row.attemptCount() == null || row.attemptCount() <= 0) {
    showError("Cannot Publish Yet", "This quiz has no attempts yet.");
    return;
}
```

This is one of the clearest examples of the interaction between UI validation and database rules. The UI prevents a bad action early, and the database still remains the final authority if a malformed request somehow bypasses the screen checks.

```
CREATE OR REPLACE TRIGGER TRG_BIU_STUDENT_VALIDATE
BEFORE INSERT OR UPDATE ON STUDENT
FOR EACH ROW
BEGIN
  IF NOT REGEXP_LIKE(:NEW.NAME, '^[A-Za-z][A-Za-z .''-]*$') THEN
    RAISE_APPLICATION_ERROR(-20002, 'Student name must contain only alphabets and basic punctuation.');
  END IF;

  IF NOT REGEXP_LIKE(:NEW.REGISTRATION_NO, '^[0-9]{9}$') THEN
    RAISE_APPLICATION_ERROR(-20003, 'Registration number must be exactly 9 digits.');
  END IF;
END;
/
```

Chapter 8: Java Backend Layer

The backend is intentionally small in conceptual terms even though it supports many operations. It uses a single connection helper, a DAO contract, a DAO implementation, and a transaction-wrapping service class. That is enough to keep the application clean without adding unnecessary complexity.

```
public class QuizManagementServiceImpl implements QuizManagementService {
    private final QuizManagementDao dao;

    private <T> T inTransaction(SqlWork<T> work) {
        try (Connection connection = DbUtil.getConnection()) {
            connection.setAutoCommit(false);
            ...
        }
    }
}
```

This service pattern is important because it protects all multi-step operations. For example, adding a question and then inserting four options should not leave the database in a partially written state. If one insert fails, the transaction can roll back and keep the quiz data consistent.

Backend Component	Job
DbUtil	Opens JDBC connections
QuizManagementDao	Defines all required data operations
QuizManagementDaoImpl	Implements Oracle queries and stored calls
QuizManagementServiceImpl	Runs operations in transactions
Custom exceptions	Translate low-level failures into readable errors

Chapter 9: JavaFX Frontend Layer

The JavaFX layer is user-focused and role-specific. Screens are not overloaded with unrelated controls. Instead, the interface presents the exact actions the current role needs. This keeps the app usable and makes demonstrations easier to follow.

Screen	Controller	Purpose
auth-choice.fxml	AuthChoiceController	Role selection
teacher-login.fxml	TeacherLoginController	Teacher sign-in
student-login.fxml	StudentLoginController	Student sign-in
admin-login.fxml	AdminLoginController	Admin sign-in
teacher-home.fxml	TeacherHomeController	Teacher dashboard
student-dashboard.fxml	StudentDashboardController	Student dashboard
quiz-attempt.fxml	QuizAttemptController	Timed quiz attempt
teacher-review.fxml	TeacherReviewController	Publish and review

The navigator is what makes the UI feel cohesive. It loads the FXML views, applies the shared CSS, and adds the selected theme stylesheet. A small fade transition makes the screen changes feel smoother without turning the app into a visually noisy interface.

```
private static void loadAndShow(String resourcePath, String title, double width, double
height) {
    FXMLLoader loader = new
FXMLLoader(QuizManagementApp.class.getResource(resourcePath));
    Parent root = loader.load();
    Scene scene = new Scene(root, width, height);

scene.getStylesheets().add(QuizManagementApp.class.getResource("/com/collegequiz/css/app.
css").toExternalForm());
    ...
}
```

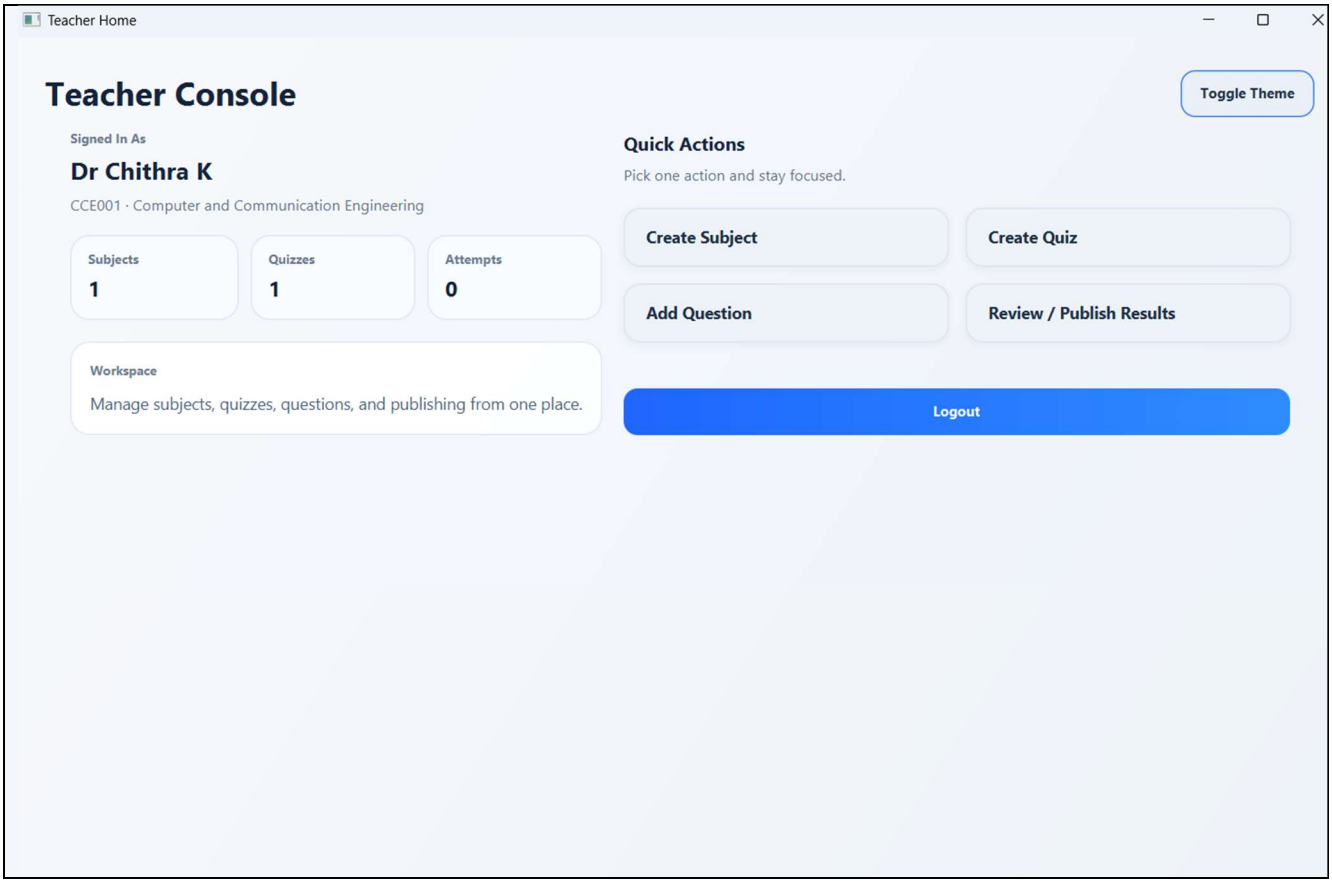
Chapter 10: Screens, Workflows, and Code Snippets

The application launches through a short bootstrap class and then delegates navigation to AppNavigator. That is a good design because it keeps startup logic separate from the rest of the user experience. The first screen is always the role choice entry point, which matches the way the project is described in the documentation.

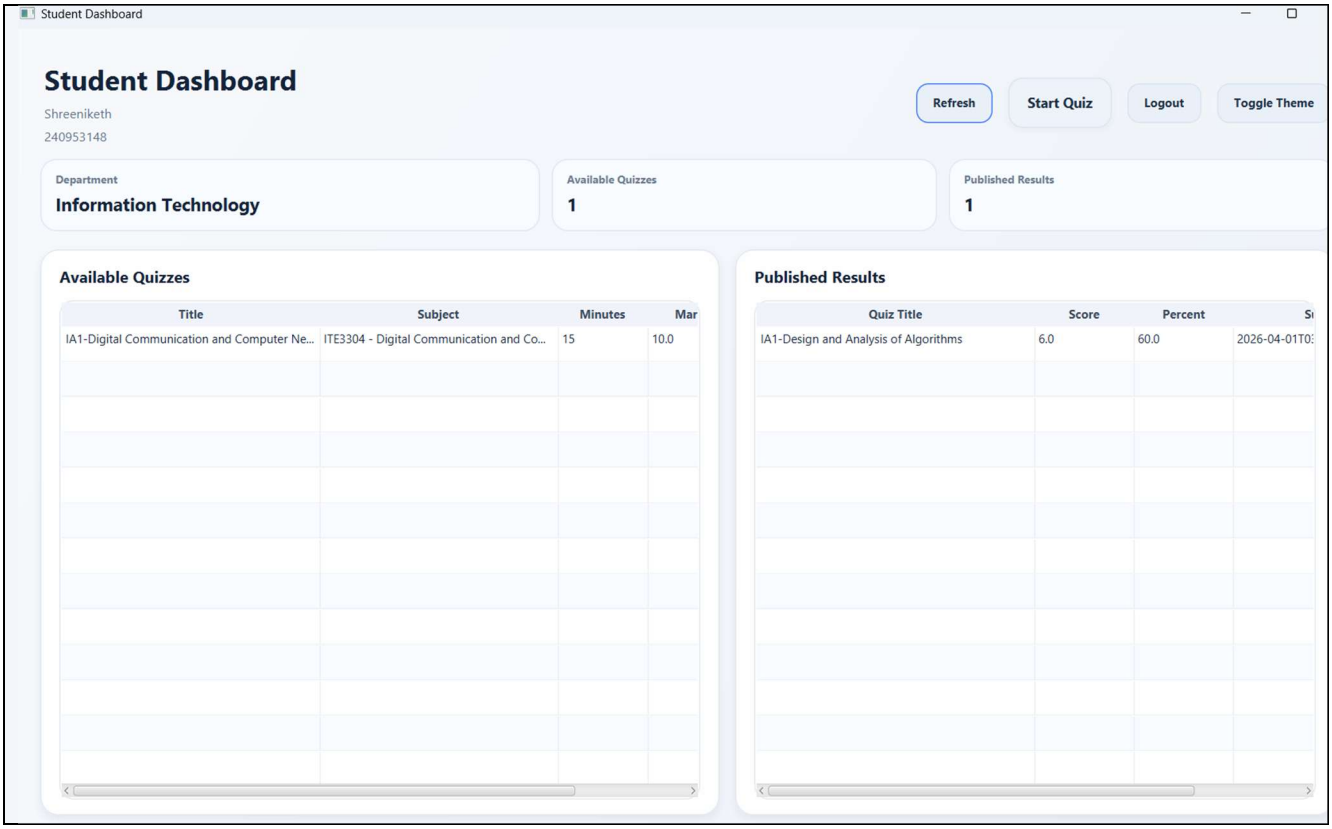
The teacher review screen is one of the most important screens in the project because it directly controls result publication. The table view shows title, subject, attempts, marks, and publication state, and the action buttons let the teacher publish, unpublish, or delete a quiz after confirming the action.

```
TeacherDashboardRow row = dashboardTable.getSelectionModel().getSelectedItem();
if (row == null) {
    showError("Select Quiz", "Choose a quiz first.");
    return;
}
service.publishResults(row.quizId(), AppSession.getLoggedInTeacherId());
```

That snippet shows a simple but important pattern. The controller checks the selected row, validates the action, and only then calls the service layer. This is exactly the kind of defensive UI logic that prevents unnecessary database errors while keeping the code easy to understand.



The student workflow is equally important. A student sees quizzes and results separately, which avoids confusing in-progress assessment with published scoring. The quiz attempt screen then becomes a controlled timed environment instead of a generic form.

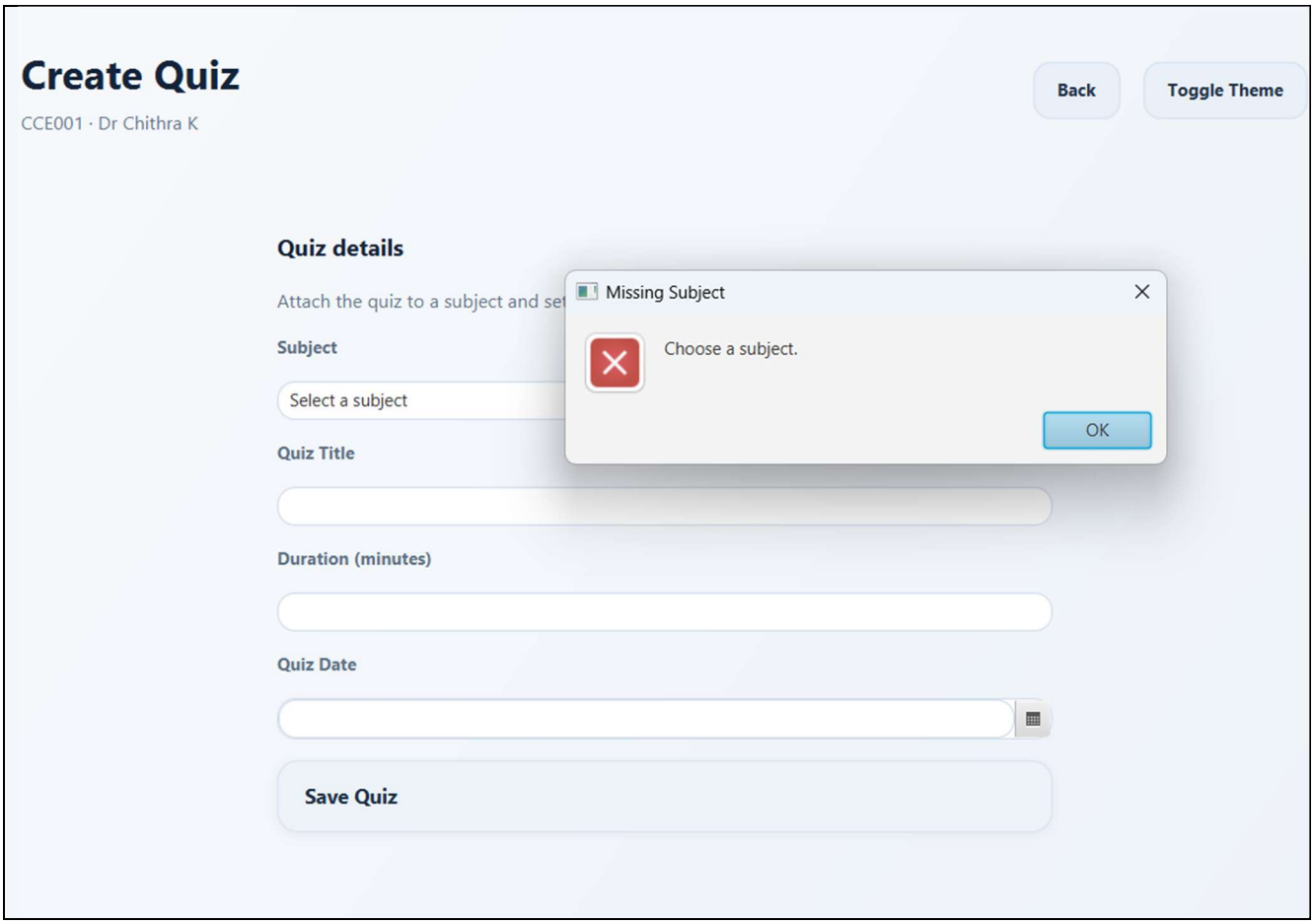


Chapter 11: Validation, Security, and Data Integrity

The project relies on validation at multiple levels. The UI checks obvious mistakes early, the backend wraps changes in transactions, and the database enforces integrity constraints. That layered approach is what makes the system robust enough for multi-user academic use.

Rule	Layer	Example
One attempt per quiz	Database	A student cannot create duplicate attempts
One correct option	Database / PL/SQL	Each question has a single correct answer
Teacher publish guard	UI and database	A quiz must have attempts before publishing
Result visibility	Database view and UI	Scores stay hidden until publication

The login design also contributes to security. Each role has a distinct credential pattern. This makes the app easier to use and harder to misuse because the student, teacher, and admin flows do not share a single confusing login pathway.



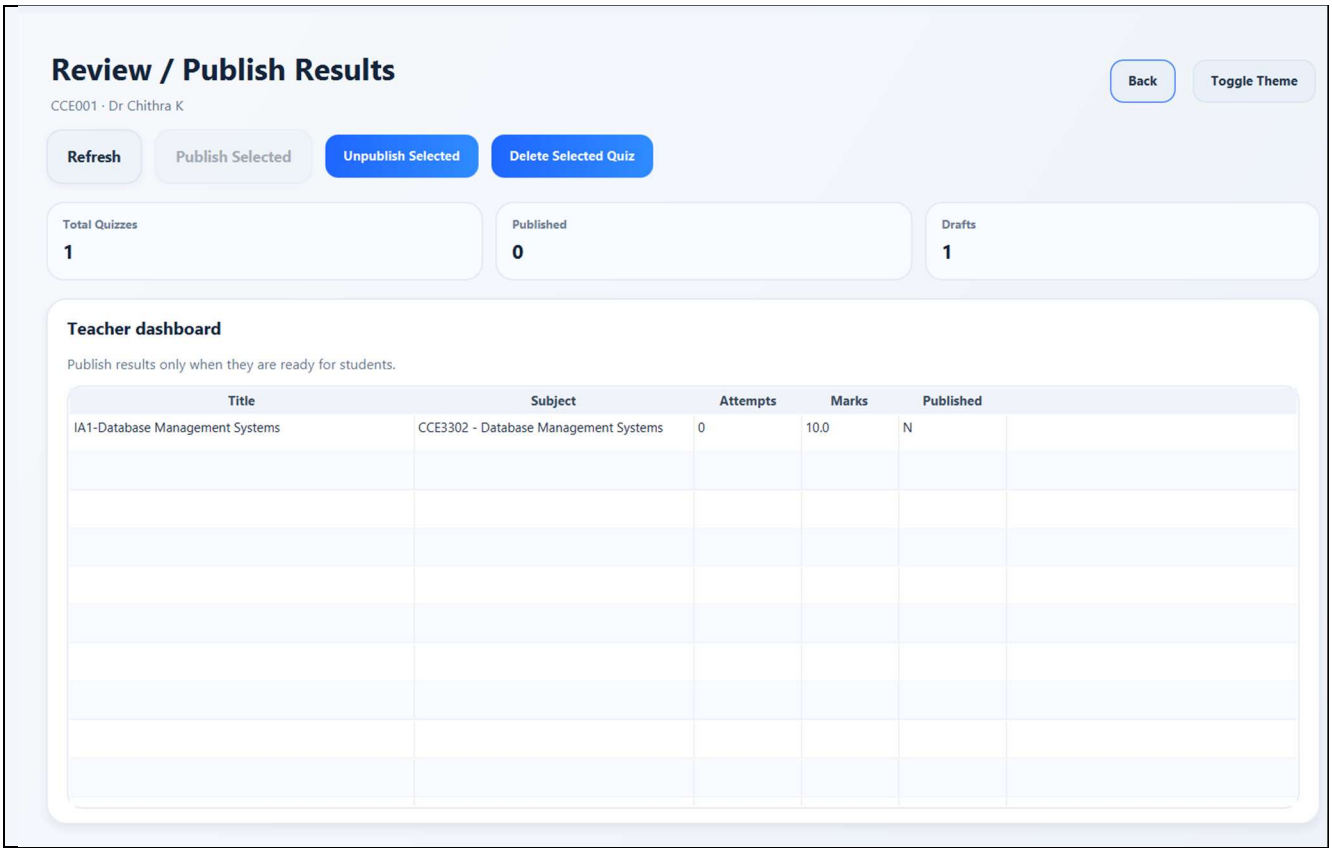
Chapter 12: Testing and Deployment

Testing the application begins with the database script and ends with the UI workflow. The application should be able to start from a clean database, accept logins, create sample data, execute a quiz attempt, and publish results without manual repair steps in between.

1. Start Oracle Database XE and confirm that SQL*Plus can connect.
2. Run the step1_quiz_management_oracle.sql script to create the schema and seed data.
3. Confirm that the JDBC settings in DbUtil match the local database.
4. Launch the app with mvn javafx:run.
5. Test each login flow with sample credentials.
6. Create and attempt a quiz, then publish results as a teacher.

Because the project is built with Maven, deployment is straightforward. The database setup script can be rerun as needed, and the JavaFX application can be started from the same project root each time. That keeps the project practical for demos and repeated testing.

Chapter 13: Conclusion

The College Quiz Management System shows how a database-oriented desktop application can be organized cleanly across Oracle, Java, and JavaFX. The database enforces the quiz rules, the backend manages transactions and Oracle calls, and the UI gives each role a simple and understandable workflow.

The report follows the PDF reference style by using chapter-based organization, table-driven summaries, and explicit placeholders where screenshots should appear. It also uses short code snippets to explain the logic behind navigation, database access, and publishing behavior. That makes the document more concrete and easier to discuss in a viva or presentation.